

# Weaviate 向量数据库上机实验报告

姓名：凌晨      学号：3125351112

## 一、实验相关概念

Weaviate 是一个开源的向量数据库 (vector database)，专门用于高效地存储、搜索和管理向量化的数据。它的核心理念是将内容（文本、图像等）转化为高维向量，然后通过相似度搜索（如余弦相似度、点积等）来实现快速、准确的语义搜索或推荐。相比传统的关系型数据库或全文检索引擎，Weaviate 对非结构化数据尤其友好，能够满足许多自然语言处理、计算机视觉等场景下的需求。

在实际使用中，Weaviate 可以将你的文本、图像等内容进行自动向量化（或使用已有的向量表征），然后把这些向量保存到数据库中。当需要进行语义搜索时，例如用一段查询文本来匹配数据库中的相似内容，Weaviate 会根据向量相似度给出结果，往往能得到比关键词匹配更丰富、更准确的检索结果。

此外，Weaviate 也提供了 RESTful API、GraphQL 以及 Python/JS 客户端等多种接口来方便开发者集成。通过其可扩展的架构，你可以很容易地将 Weaviate 部署到云服务环境中，并支持水平扩展，以应对大规模数据和高并发查询需求。与常见的向量检索库（如 Faiss、Annoy 等）相比，Weaviate 更像是一个“服务”或“数据库引擎”，不仅提供索引和检索功能，还包含了数据管理、安全认证、数据库事务等方面的功能，让你可以在同一个平台里完成更多端到端的开发工作。

在深度学习应用不断扩张的时代，随着文本、音视频、图像和传感器数据的多样化，向量数据库作为底层存储和检索工具越来越受到关注。Weaviate 便是这一领域的一个代表，通过简化向量数据管理和搜索操作，为构建各种人工智能应用提供了稳定、高性能的支持。

## 二、实验环境安装与配置

在本实验中，实验环境的搭建与配置包含以下几个关键步骤：

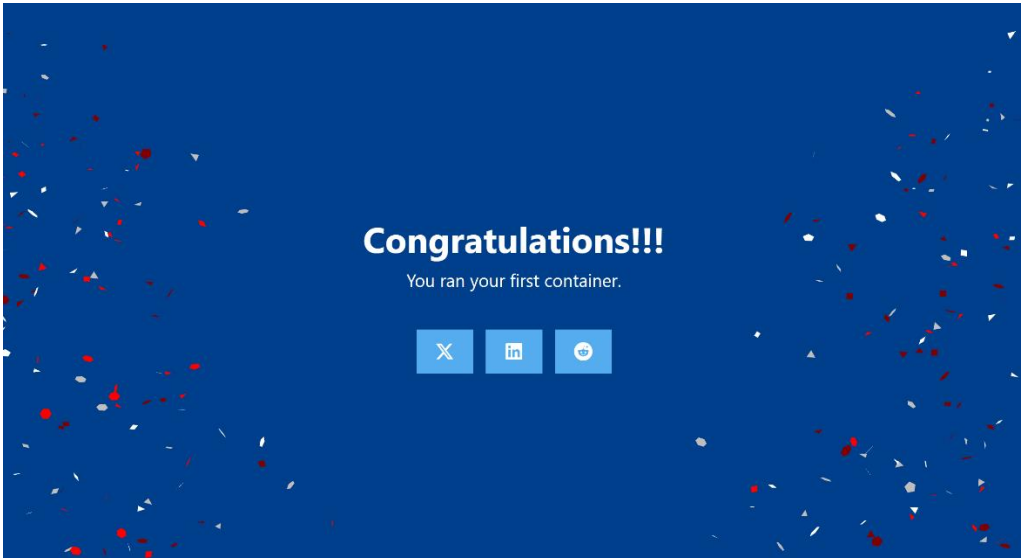
### Python 及 Jupyter Notebook 环境

通过“Python 数据库上机实验”掌握了 Python 编程语言及 Jupyter Notebook 环境的基本操作。确保本地安装了 Python，并通过 pip 安装了 Jupyter Notebook 相关依赖，为后续的代码运行提供支持。

### Docker Desktop 安装与配置

从 Docker 官方网站 下载 Docker Desktop Installer，并完成其安装与配置。安装后，确保 Docker Desktop 容器运行环境启动正常。为验证容器环境是否正常运行，实验中使用了课程扩展实验资料中的 welcome-to-docker-main.zip 压缩包。解压后，根据 README.md

文件中的说明执行 `docker run -d -p 8088:80 --name welcome-to-docker docker/welcome-to-docker` 打开浏览器，访问 `http://localhost:8088`，显示测试页面，表明 Docker 环境正常运行。



## Weaviate 向量数据库安装与配置

根据课程提供的“向量数据库 Weaviate 使用教程”，采用 Docker 容器运行环境安装 Weaviate。具体步骤如下：从官网下载构建好的 `docker-compose.yaml` 文件，并通过 `docker-compose up -d` 构建镜像和容器。打开浏览器，访问 `http://127.0.0.1:8888/lab`，能够进入 Jupyter Lab 环境，表明 Weaviate 容器运行正常。

```
C:\Windows\System32\cmd.exe
C:\Users\guoyiyang>curl -o docker-compose.yaml "https://configuration.weaviate.io/v2/docker-compose/docker-compose.yaml?modules=standard&runtime=docker-compose&weaviate_version=v1.28.2&weaviate_volume=no-volume"
% Total % Received % Xferd Average Speed Time Time Time
Dload Upload Total Spent Left Current
100 499 100 499 0 0 322 0 0:00:01 0:00:01 --:--:-- 326

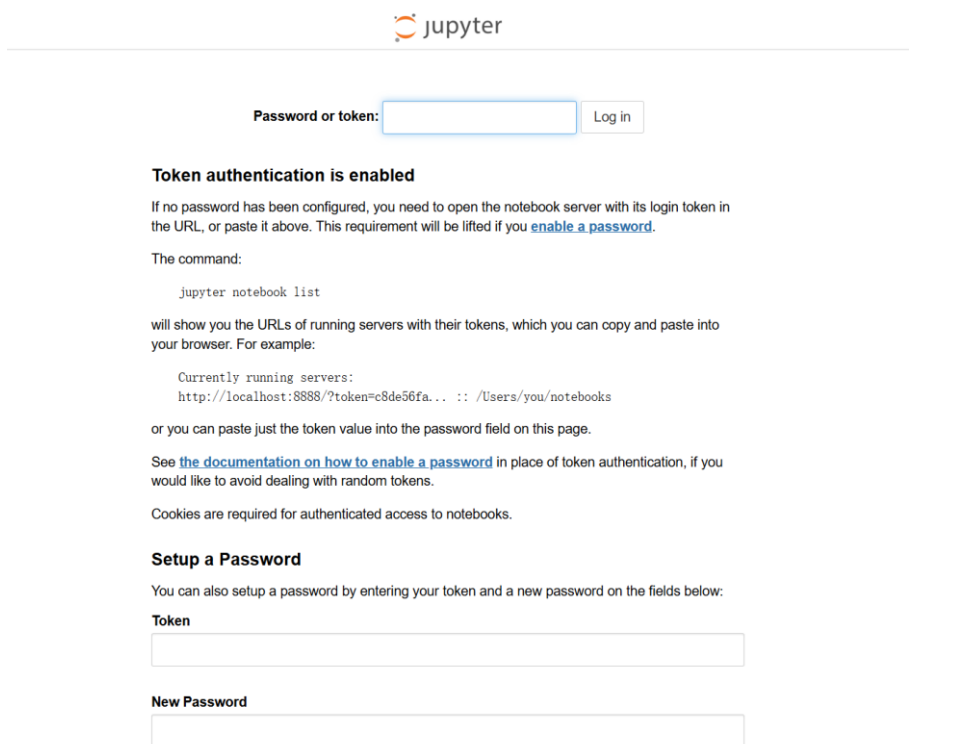
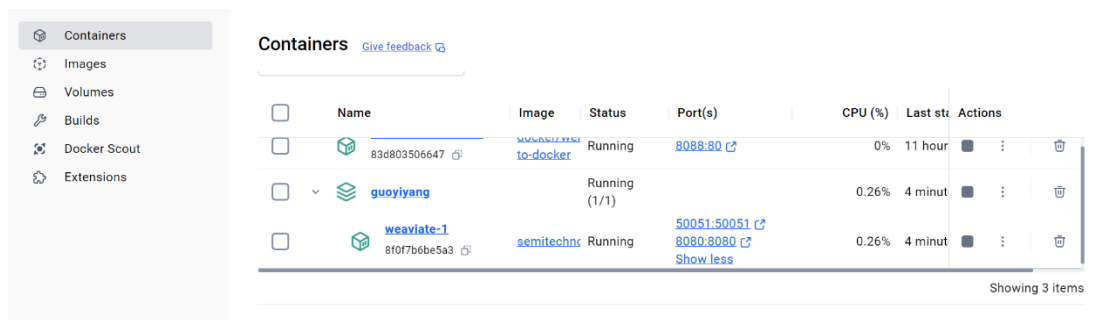
C:\Users\guoyiyang>ls -al
'ls' 不是内部或外部命令，也不是可运行的程序
或批处理文件。

C:\Users\guoyiyang>docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
83d803506647 docker/welcome-to-docker "/docker-entrypoint..." 11 hours ago Up 11 hours 0.0.0.0:8088->80/tcp welcome-to-docker

C:\Users\guoyiyang>docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
83d803506647 docker/welcome-to-docker "/docker-entrypoint..." 11 hours ago Up 11 hours 0.0.0.0:8088->80/tcp welcome-to-docker

C:\Users\guoyiyang>docker-compose up -d
[+] Running 8/8
  ✓ weaviate Pulled 22.2s
  ✓ 38a8210d887e Pull complete 1.8s
  ✓ 00fca6b36eef Pull complete 2.0s
  ✓ e94445e67f6a Pull complete 5.2s
  ✓ 674b8b4aca8f Pull complete 5.2s
  ✓ 98e6e5efd6f3 Pull complete 5.4s
  ✓ 2400113185a0 Pull complete 5.5s
  ✓ 9463f7d075a6 Pull complete 5.5s
[+] Running 2/2
  ✓ Network guoyiyang_default Created 0.0s
  ✓ Container guoyiyang-weaviate-1 Started 0.6s

C:\Users\guoyiyang>docker ps -a
CONTAINER ID IMAGE NAMES COMMAND CREATED STATUS PORTS
8f0f7b6be5a3 cr.weaviate.io/semitechnologies/weaviate:1.28.2 "/bin/weaviate --hos..." About a minute ago Up About a minute 0.
0.0.0:8080->8080/tcp, 0.0.0.0:50051->50051/tcp guoyiyang-weaviate-1
83d803506647 docker/welcome-to-docker "/docker-entrypoint..." 11 hours ago Up 11 hours 0.
```



## 验证配置环境

在 `/weaviate-jupyter-main/notebook` 目录下使用 Jupyter Notebook 打开 `Weaviate+Resnet18_Vector_Similarity.ipynb` 文件，并成功运行其中代码，验证配置的 Weaviate 环境可以正常处理向量搜索相关任务。

通过上述步骤，完成了实验所需的 Python 编程环境、Docker 容器环境以及 Weaviate 向量数据库的安装与配置，为后续实验操作奠定了基础。

## 三、实验操作程及结果

熟悉实验环境：

jupyter Weaviate+Resnet18\_Vector\_Similarity 最新检查点: 2023/11/10 (已自动保存) Logout

File Edit View Insert Cell Kernel Widgets Help 不可信 Python 3 (ipykernel)

```
In [1]: import torch
import torch.nn as nn
import torchvision.models as models
import torchvision.transforms as transforms
from torch.autograd import Variable
from PIL import Image
import matplotlib.pyplot as plt
import numpy as np
%matplotlib inline

In [2]: class FeatureVector(object):
    def __init__(self):
        self.model = models.resnet18(pretrained=True)
        self.model.eval()
        self.layer = self.model._modules.get("avgpool")
        self.cos = nn.CosineSimilarity(dim=1, eps=1e-6)
        self.transform_pipeline = transforms.Compose([transforms.Resize((224, 224)),
                                                    transforms.ToTensor(),
                                                    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                                                                           std=[0.229, 0.224, 0.225])])

    def get_vector(self, img):
        t_img = Variable(self.transform_pipeline(img)).unsqueeze(0)
        feat_vec = torch.zeros(512)

        def copy_data(m, i, o):
            feat_vec.copy_(o.data.squeeze())

        h = self.layer.register_forward_hook(copy_data)
        self.model(t_img)
        h.remove()

        return feat_vec

    def similarity(self, a, b):
        return self.cos(a.unsqueeze(0), b.unsqueeze(0))
```

jupyter Weaviate+Resnet18\_Vector\_Similarity 最新检查点: 2023/11/10 (更改未保存) Logout

File Edit View Insert Cell Kernel Widgets Help 可信 Python 3 (ipykernel)

```
../images/person2.jpg',
../images/stones.jpg']

In [13]: image_file_name_path_mapping = { x.split("/")[-1]: x for x in image_filepaths}
image_file_name_path_mapping

Out[13]: {'cat1.jpg': '../images/cat1.jpg',
'cat2.jpg': '../images/cat2.jpg',
'chair.jpg': '../images/chair.jpg',
'dog1.jpg': '../images/dog1.jpg',
'dog2.jpg': '../images/dog2.jpg',
'dog3.jpg': '../images/dog3.jpg',
'dog4.jpg': '../images/dog4.jpg',
'dog5.jpg': '../images/dog5.jpg',
'person1.jpg': '../images/person1.jpg',
'person2.jpg': '../images/person2.jpg',
'stones.jpg': '../images/stones.jpg'}

In [8]: image_mappings = {x.split("/")[-1]:Image.open(x) for x in image_filepaths}
image_mappings

NameError                                Traceback (most recent call last)
Cell In[8], line 1
----> 1 image_mappings = {x.split("/")[-1]:Image.open(x) for x in image_filepaths}
      2 image_mappings

Cell In[8], line 1. in <dictcomp>(.0)
----> 1 image_mappings = {x.split("/")[-1]:Image.open(x) for x in image_filepaths}
      2 image_mappings

NameError: name 'Image' is not defined

In [14]: image_mappings.keys()

Out[14]: dict_keys(['cat1.jpg', 'cat2.jpg', 'chair.jpg', 'dog1.jpg', 'dog2.jpg', 'dog3.jpg', 'dog4.jpg', 'dog5.jpg', 'person1.jpg', 'person2.jpg',
'stones.jpg'])

In [8]: image_mappings['dog2.jpg']

Out[8]: 
```

```
In [2]: def compare(a, b):
        plt.figure()
        plt.subplot(1, 2, 1)
        plt.imshow(a)
        plt.subplot(1, 2, 2)
        plt.imshow(b)
        a_v = feat_vec.get_vector(a)
        b_v = feat_vec.get_vector(b)
        print('Similarity: {}'.format(feat_vec.similarity(a_v, b_v)))

In [ ]: compare(image_mappings['dog2.jpg'], image_mappings['dog3.jpg'])
        Similarity: tensor([0.6009])

In [ ]: compare(image_mappings['dog1.jpg'], image_mappings['dog3.jpg'])
        Similarity: tensor([0.5898])

In [14]: import weaviate

ImportError                                Traceback (most recent call last)
Cell In[14], line 1
----> 1 import weaviate

File C:\ProgramData\anaconda3\Lib\site-packages\weaviate\__init__.py:3
      1 from importlib.metadata import version, PackageNotFoundError
----> 3 from . import classes
      5 try:
      6     __version__ = version("weaviate")

ImportError: cannot import name 'classes' from partially initialized module 'weaviate' (most likely due to a circular import) (C:\ProgramData\anaconda3\Lib\site-packages\weaviate\__init__.py)

1 [1]: import sys
import weaviate

print("Python executable:", sys.executable)
print("weaviate client version:", weaviate.__version__)

Python executable: C:\ProgramData\anaconda3\python.exe
weaviate client version: 3.26.7

1 [2]: _HOST = "localhost"
_PORT = "8080"
print(weaviate.__version__)
weaviate_url = f"http://{_HOST}:{_PORT}"

client = weaviate.Client(url=weaviate_url)
client.is_ready()

3.26.7

Out[2]: True

1 [3]: class_name = "Images"

1 [4]: client.schema.delete_all()

1 [5]: image_class_schema = {
        # name of the class
        "class": class_name,
        # a description of what this class represents
        "description": "An Image class to store the image paths and their vectors",
        # class properties
        "properties": [
```

## Exercise1

连接 Weaviate 数据库

You can check the cluster at the given URL. You can check the meta information at `<weaviate_url>/v1/meta`, its data schema at `<weaviate_url>/v1/schema` and the data objects at `<weaviate_url>/v1/objects`. You can also see this cluster on your homepage on [console.semi.technology](#) (when you are logged in).

```
[24]: In [1]: _HOST = "localhost"
        _PORT = "8080"
        print(weaviate.__version__)
        weaviate_url = f"http://{_HOST}:{_PORT}"

        client = weaviate.Client(url=weaviate_url)
        client.is_ready()

3.4.2

Out[24]: True
```

下载实验数据

```
In [46]: data = []

# for example:
data += get_articles_from_newspaper('https://www.wired.com/')
# add more newspapers if you like!

https://www.wired.com/: 100%
██ 100/100 [02:20:00:00, 1.40s/it]
```

```
In [46]: data
Out[46]: [{"id": "50e53e87-212a-387f-a8d0-7b4de43e9e76",
  "title": "I Used AI to Do All of My Holiday Shopping",
  "summary": "Over the last few weeks, I've offloaded my gift shopping to Perplexity AI, OpenAI's ChatGPT, Google's Gemini, Anthropic's Claude, and Amazon's Rufus to find out whether I could finally delegate one of the tasks I despise the most to AI. I set the generative AI bots loose and burned planet Earth in a quest to find the perfect baking equipment. None of the AI apps I tested has a feature specifically designed for shopping. Perplexity AI, a well-funded generative AI search startup that has been criticized for allegedly lifting content from news publishers, rolled out a new service last month called Buy with Pro, which comes with a $20-per-month subscription to Perplexity Pro. Buywith Pro is described as a "first-of-its-kind AI commerce experience" that promises to make shopping online "10X more easy and fun." This immediately sounded like an AI hallucination to me, because online shopping isn't fun in the first place.",
  "authors": ["Lauren Goode",
    "Nena Farrell",
    "Scott Gilbertson",
    "Michael Calore",
    "Lisa Wood Shapiro",
    "Jaina Grey",
    "Louryn Strampe",
    "Parker Hall",
    "Simon Hill"]}]
```

定义另外两个类，并包含以下属性：**"Author"**（作者）类具有属性：**"name"**（名字）和**"wroteArticles"**（撰写的文章）。**"Category"**（类别）类具有一个属性：**"name"**（名字）。

```

        {
            "name": "hasAuthors",
            "dataType": ["Author"],
            "description": "The authors this article has",
        },
        {
            "name": "hasCategory",
            "dataType": ["Category"],
            "description": "The category of this article",
        }
    ],
    {
        "class": "Author",
        "description": "An author class to store authors",
        "properties": [
            {
                "name": "name",
                "dataType": ["string"],
                "description": "Name of the author",
            },
            {
                "name": "wroteArticles",
                "dataType": ["Article"],
                "description": "Articles of the author",
            }
        ],
    },
    {
        # Write the Category class here
        # Write the Category class here
        "class": "Category",
        "description": "An category class to store news categories",
        "properties": [
            {
                "name": "name",
                "dataType": ["string"],
                "description": "Name of the category",
            }
        ],
    },
]
}

```

In [50]: `client.schema.create(schema)`

Lets get the schema from weaviate and look what was created.

In [51]: `# helper function`  
`def prettify(json_dict):`  
 `print(json.dumps(json_dict, indent=2))`

In [52]: `prettify(client.schema.get())`

```

{
  "classes": [
    {
      "class": "Article",
      "description": "An Article class to store the article summary and its authors",
      "invertedIndexConfig": {
        "bm25": {
          "b": 0.75,
          "k1": 1.2
        },
        "cleanupIntervalSeconds": 60,
        "stopwords": {
          "additions": null,
          "preset": "en",
          "removals": null
        }
      },
      "multiTenancyConfig": {
        "autoTenantActivation": false,

```

You can always delete your entire schema with `client.schema.delete_all()`.

## Exercise4

补全 `add_author()`和 `add_references()`两个函数

```

    return article_id

def add_author(batch: Batch, author_name: str) -> str:

    author_object = {'name': author_name}

    # generate an UUID for the Author
    author_id = generate_uuid5(author_name)

    # add author to the batch
    # EXERCISE: call here the batch.add_data_object function to add the author to the batch
    # todo
    batch.add_data_object(

        data_object=author_object,
        class_name="Author",
        uuid=author_id
    )
    return author_id

def add_references(batch: Batch, article_id: str, author_id: str) -> None:
    # add references to the batch
    ## Author -> Article
    batch.add_reference(
        from_object_uuid=author_id,
        from_object_class_name="Author",
        from_property_name="wroteArticles",
        to_object_uuid=article_id
    )

    ## Article -> Author
    # EXERCISE: call here the batch.add_reference function to add the article->author reference
    # todo
    batch.add_reference(
        from_object_uuid=article_id,
        from_object_class_name="Article",
        from_property_name="hasAuthors",
        to_object_uuid=author_id
    )

```

## Exercise5

### Weaviate 查询

1

#### 2.5.2 Query data using python

You can use all GraphQL functions also with the Weaviate Python client. For reference, [check the docs](#). Some examples:

##### 1. Get a list of article names

```

In [56]: result = client.query.get("Article", ["title"]).do()
         prettify(result)

{
  "data": {
    "Get": {
      "Article": [
        {
          "title": "Nosferatu\u2019s Resurrection Was a Long Time Coming"
        },
        {
          "title": "The Fight Against Surveillance Capitalism: A Conversation with Meredith Whittaker"
        },
        {
          "title": "Kash Patel Is the Hero QAnon Has Been Waiting For"
        },
        {
          "title": "2025 Will Be Smart Glasses All the Way Down"
        },
        {
          "title": "Kamala Harris Answers The Web's Most Searched Questions"
        }
      ]
    }
  }
}

```

2 和 3



## 2. Get the number of articles present

```
In [59]: result = client.query.aggregate("Article") \
        .with_fields('meta { count }') \
        .do()
        prettify(result)

{
  "data": {
    "Aggregate": {
      "Article": [
        {
          "meta": {
            "count": 100
          }
        }
      ]
    }
  }
}
```

## 3. Get the average, minimum and maximum word count of all articles

```
In [60]: result = client.query.aggregate("Article") \
        .with_fields('wordCount {maximum mean minimum}') \
        .do()
        prettify(result)

{
  "data": {
    "Aggregate": {
      "Article": [
        {
          "wordCount": {
            "maximum": 156,
            "mean": 89.61,
            "minimum": 21
          }
        }
      ]
    }
  }
}
```

截图工具  
屏幕截图已  
自动保存

4

## 4. Get article titles and cross-ref authors

```
In [61]: query = "{Get {Article {title hasAuthors {... on Author {name }}}}"}"
        result = client.query.raw(query)
        prettify(result)

{
  "data": {
    "Get": {
      "Article": [
        {
          "hasAuthors": [
            {
              "name": "Angela Watercutter"
            },
            {
              "name": "Ali Winston"
            },
            {
              "name": "Wired Readers"
            },
            {
              "name": "Matt Kamen"
            }
          ]
        }
      ]
    }
  }
}
```

5 和 6

#### 5. Get the title, id and vector of the first article

```
In [62]: result = client.query.get("Article", ["title", "_additional (id vector) "]).with_limit(1).do()
prettify(result)

{
  "data": {
    "Get": {
      "Article": [
        {
          "_additional": {
            "id": "000777fb-f786-37a8-a357-aa473817elf8",
            "vector": []
          },
          "title": "Nosferatu\u2019s Resurrection Was a Long Time Coming"
        }
      ]
    }
  }
}
```

#### 6. Semantic search: get articles that are near "animal" and return the title, summary and semantic certainty

```
In [63]: nearText = {
  "concepts": ["animal"]
}

client.query.get("Article", ["title", "summary", "_additional (certainty) "]).with_near_text(nearText).do()

Out[63]: {'errors': [{'locations': [{'column': 14, 'line': 1}],
  'message': 'Unknown argument "nearText" on field "Article" of type "GetObjectsObj". Did you mean "nearObject" or "nearVector"?',
  'path': None}]}
```

7

#### 7. Semantic search: get articles that are near "music" and return the title, summary and semantic certainty, if certainty is above 0.8

```
In [64]: nearText = {
  "concepts": ["music"],
  "certainty": 0.8
}

client.query.get("Article", ["title", "_additional (certainty) "]).with_near_text(nearText).do()

Out[64]: {'errors': [{'locations': [{'column': 14, 'line': 1}],
  'message': 'Unknown argument "nearText" on field "Article" of type "GetObjectsObj". Did you mean "nearObject" or "nearVector"?',
  'path': None}]}
```

8

#### 8. You can also combine semantic and scalar filters:

```
In [65]: query = """{
  Get {
    Article (
      where: {
        operands: [
          {
            path: ["hasAuthors"],
            valueInt: 2
            operator: GreaterThan
          },
          {
            path: ["wordCount"],
            valueInt: 100
            operator: GreaterThan
          }
        ],
        operator: And
      }
      nearText: {
        concepts: ["animals"]
      }
    ) {
      title
      wordCount
      hasAuthors {
        ... on Author {
          name
        }
      }
    }
  }
}"""

result = client.query.raw(query)
prettify(result)

{
  "errors": [
    {
      "locations": [
        {
          "column": 7,
          "line": 16
        }
      ],
      "message": "Unknown argument \"nearText\" on field \"Article\" of type \"GetObjectsObj\". Did you mean \"nearObject\" or \"nearVect",
      "path": null
    }
  ]
}
```

9.

#### 9. Use the Question Answering module:

```
In [66]: ask = {
    "question": "Which earplugs should I buy?",
    "rerank": True,
    "properties": ["summary"]
}

client.query.get("Article", [{"title", "_additional {answer {hasAnswer certainty property result startPosition endPosition} }"}]).with_ask(ask)

Out[66]: {'errors': [{'locations': [{'column': 23, 'line': 1}],
  'message': 'Unknown argument "ask" on field "Article" of type "GetObjectsObj".',
  'path': None},
  {'locations': [{'column': 116, 'line': 1}],
  'message': 'Cannot query field "answer" on type "ArticleAdditional".',
  'path': None}]}
```

Congratulations! Now you know how to start a Weaviate cluster and use the Python client to upload and query data!

We've set up a very basic vector search application. For more features, check out the documentation. For example, you can add [Question Answering](#), do [multi-modal search](#), start automatic [classifications](#), and much more!

You can join the [Weaviate Slack channel](#) to interact with other Weaviate users and ask questions or give feedback :)

## Exercise6

### 自定义新闻类型以及查询结果

#### Exercise 6: Zero shot classification to assign categories to articles

Follow the steps below.

First, define some categories. You can choose yourself which categories you want to add. Keep in mind that every article will be assigned to one of the categories (no article will be left), with this simple form of classification. Categories can be for example "Business", "Sports", "Environment", etc.

```
In [73]: # define the categories, add as many as you want!
categories = ["Business", "Arts", "Technology", "Sports"]

In [75]: # add the categories as data objects
for category in categories:
    client.batch.add_data_object({"name": category}, "Category", generate_uuid5(category))
client.batch.create_objects()

Out[75]: [{'class': 'Category',
  'creationTimeUnix': 1735197225627,
  'id': 'b3ab2519-219a-503d-9907-67bb2946d764',
  'lastUpdateTimeUnix': 1735197225627,
  'properties': {'name': 'Business'},
  'deprecations': None,
  'result': {'status': 'SUCCESS'}},
  {'class': 'Category',
  'creationTimeUnix': 1735197225627,
  'id': '5989f4b1-b17f-525b-8073-0511c9af605c',
  'lastUpdateTimeUnix': 1735197225627,
  'properties': {'name': 'Arts'},
  'deprecations': None,
  'result': {'status': 'SUCCESS'}},
  {'class': 'Category',
  'creationTimeUnix': 1735197225627,
  'id': '029fa9b2-aac9-5b72-af68-e89ec094c601',
  'lastUpdateTimeUnix': 1735197225627,
  'properties': {'name': 'Technology'},
  'deprecations': None,
  'result': {'status': 'SUCCESS'}},
  {'class': 'Category',
  'creationTimeUnix': 1735197225627,
  'id': 'fce3516e-be3d-5216-ba32-a2522f225fcl',
  'lastUpdateTimeUnix': 1735197225627,
  'properties': {'name': 'Sports'},
  'deprecations': None,
  'result': {'status': 'SUCCESS'}}}]
```

```
In [76]: # perform the classification
client.classification.schedule()\
    .with_type("zeroshot")\
    .with_class_name("Article")\
    .with_classify_properties(["hasCategory"])\
    .with_based_on_properties(["summary"])\
    .with_wait_for_completion()\
    .do()

Out[76]: {'basedOnProperties': ['summary'],
          'class': 'Article',
          'classifyProperties': ['hasCategory'],
          'error': 'classification failed: no classes to be classified - did you run a previous classification already?',
          'id': '149f313f-b7b9-4ad1-9e36-e177b2f43b86',
          'meta': {'completed': '0001-01-01T00:00:00.000Z',
                    'started': '2024-12-26T07:14:07.493Z'},
          'status': 'failed',
          'type': 'zeroshot'}
```

Once the classification is complete and successful, you can check if your articles now have a category assigned, using a GraphQL query.

## 四、实验总结

本次实验中，我成功完成了开源向量数据库 **Weaviate** 的环境配置，并基于该数据库实施了一系列向量数据操作，最终构建了一个简易的向量搜索应用。实验的过程中，我学习并掌握了将向量化数据存储到 **Weaviate** 的方法，同时熟悉了其强大的相似度检索功能，用于快速定位相关向量。此外，我还尝试利用 **Weaviate** 进行向量数据的聚类分析，并初步实践了构建推荐系统的思路。

通过这次实验，我进一步加深了对向量数据库基本原理和操作方式的理解，尤其是在实际应用场景中对其高效处理复杂数据的能力有了全新的认知。这次实验让我体会到向量数据库在智能化应用开发中的核心作用，并为未来探索数据驱动型应用设计积累了宝贵的经验和技能储备。